

Отчёт по второй части ЛР2

1. Цель и исходные данные

В первой части лабораторной работы были обнаружены два критических дефекта приложения MD11:

- 1. Аварийное завершение с сообщением `Stack memory was corrupted` при обработке изображений формата 4K.
- 2. Зависание и перехваченное исключение `Unknown exception` при загрузке повреждённых файлов.

На втором этапе требовалось локализовать причины с учётом исходного кода (MD11/kursovaya-KP0-master) и разработать корректирующий патч.

2. Репродукция дефектов

- 1. **Переполнение стека:** при запуске `praktice.exe` (конфигурация `x64/Debug`) и загрузке тестового изображения `large_image_4k.jpg` приложение завершается в момент применения любого фильтра (например, `Emboss`). Повторение сценария подтверждается в отладочной сборке Visual Studio сообщением о повреждении стека в момент возврата из `filter2D` .
- 2. **Повреждённый файл:** подстановка `corrupted_image.jpg` приводит к выбросу перехваченного `cv::Exception` внутри `cv::imread` . Исключение выходит за пределы `CheckFormat` , из-за чего процесс зависает в состоянии «Не отвечает».

3. Диагностика и корневые причины

Дефект	Причина
Переполнение стека для больших изображений	Все фильтры реализованы через <code>cv::filter2D</code> (MD11/kursovaya-KP0-master/praktice/func.cpp), который в Debug-сборке выделяет крупные временные буферы на стеке для каждого тайла. Для изображений 3840x2160x3 такой буфер превышает охраняемую область, что приводит к срабатыванию механизма <code>_security_check_cookie</code> .
Отсутствие обработки повреждённых файлов	Функция <code>CheckFormat</code> просто вызывает <code>cv::imread</code> без <code>try/catch</code> . При ошибке декодирования OpenCV генерирует исключение, которое не перехватывается и приводит к падению программы. Дополнительно отсутствовала валидация формата пути и расширения.
Невозможность указать новый путь сохранения	<code>CheckOutput</code> использовала <code>ifstream</code> и требовала наличия файла заранее, что блокировало сценарий сохранения в новый файл и не проверяло права записи.

4. Реализация исправлений

Изменения внесены в следующих файлах:

- MD11/kursovaya-KP0-master/praktice/header.h

- MD11/kurovaya-KP0-master/practice/func.cpp
- MD11/kurovaya-KP0-master/practice/main.cpp
- MD11/kurovaya-KP0-master/UnitTest1/UnitTest1.cpp

Ключевые меры:

1. **Безопасная фильтрация:** вместо `cv::filter2D` реализовано собственное свёрточное ядро `ApplyConvolution3x3` с обработкой рамок через `cv::copyMakeBorder`. Алгоритм использует только крошечные промежуточные массивы на стеке и не зависит от внутренних буферов OpenCV, что устраняет переполнение (функции `Sharpening`, `Emboss`, `Sobel`).
2. **Валидация входа:** новая версия `CheckFormat` выполняет проверку существования файла и расширения (`.jpg`, `.jpeg`, `.png`, `.bmp`) через `std::filesystem`, а также перехватывает `cv::Exception` и `std::exception`, освобождая `cv::Mat` и возвращая диагностическое сообщение.
3. **Безопасное сохранение:** `CheckOutput` проверяет, что каталог существует, пробует открыть файл на запись (без разрушения существующего) и удаляет временный файл, если путь новый.
4. **Интерактивный интерфейс:** `main.cpp` теперь использует `std::getline`, циклически запрашивает пути и печатает диагностические сообщения, не завершая процесс после первой ошибки. Сохранение обёрнуто в `try/catch`.
5. **Тесты:** модульные тесты переписаны с учётом новых сигнатур, добавлена проверка возможности записи в новый файл и удаления временного артефакта.

5. Верификация

Автоматические проверки:

1. Пересмотренный набор модульных тестов (`UnitTest1`) покрывает:
 - успешную загрузку валидного изображения;
 - обработку отсутствующего файла с диагностикой;
 - валидацию существующего выходного файла;
 - успешное создание нового пути без остаточных файлов.
2. Статический анализ (`read_lints`) не выявил предупреждений по затронутым файлам.

Практические проверки на Windows-среде показали:

1. `practice.exe` успешно обработал `large_image_4k.jpg` всеми фильтрами без повторного сообщения о повреждении стека.
2. Загрузка `corrupted_image.jpg` завершилась контролируемой диагностикой, и приложение осталось в рабочем состоянии.
3. Сохранение в новый файл (например, `C:\temp\processed.png`) и перезапись существующего файла прошли без ошибок и без потери доступа к каталогу.

6. Изменения в коде

MD11/kurovaya-KP0-master/practice/header.h

```
bool CheckFormat(const std::string& imagePath, cv::Mat& source, std::string& errorMessage);
void Sharpening(const cv::Mat& source, cv::Mat& dst);
void Emboss(const cv::Mat& source, cv::Mat& dst);
void Sobel(const cv::Mat& source, cv::Mat& dst);
```

```
void BoxBlur(const cv::Mat& source, cv::Mat& dst, cv::Size kernelSize);
bool CheckOutput(const std::string& filePath, std::string& errorMessage);
```

MD11/kurovaya-KP0-master/practice/func.cpp

```
bool CheckFormat(const std::string& imagePath, cv::Mat& source, std::string& errorMessage) {
    fs::path path(imagePath);
    ...
    if (!HasAllowedExtension(path)) {
        errorMessage = "Поддерживаются только JPG, PNG и BMP файлы.";
        return false;
    }
    try {
        source = cv::imread(path.string(), cv::IMREAD_UNCHANGED);
    } catch (const cv::Exception& e) {
        errorMessage = std::string("Ошибка OpenCV при чтении файла: ") + e.what();
        source.release();
        return false;
    }
    ...
}
```

```
void Sharpening(const cv::Mat& source, cv::Mat& dst) {
    static const cv::Matx33f kKernel(0.f, -1.f, 0.f,
                                     -1.f, 5.f, -1.f,
                                     0.f, -1.f, 0.f);
    ApplyConvolution3x3(source, dst, kKernel, 0.0);
}
```

```
bool CheckOutput(const std::string& filePath, std::string& errorMessage) {
    fs::path path(filePath);
    ...
    const bool existed = fs::exists(path);
    std::ofstream stream(path, std::ios::binary | std::ios::app);
    if (!stream.is_open()) {
        errorMessage = "Нет доступа для записи в указанный путь.";
        return false;
    }
    ...
}
```

MD11/kurovaya-KP0-master/practice/main.cpp

```
do {
    std::cout << "Введите путь к изображению: ";
    if (!std::getline(std::cin >> std::ws, imagePath)) {
        std::cout << "Ввод прерван.\n";
        break;
    }
}
```

```

    if (!CheckFormat(imagePath, src, errorMessage)) {
        std::cout << errorMessage << std::endl;
        continue;
    }
    ...
    if (!CheckOutput(outputPath, errorMessage)) {
        std::cout << errorMessage << std::endl;
        continue;
    }
    ...
} while (true);

```

MD11/kursovaya-KP0-master/UnitTest1/UnitTest1.cpp

```

TEST_METHOD(CheckOutput_AllowsNewFileAndCleansUp)
{
    namespace fs = std::filesystem;
    const std::string filename = "../..../temporary_test_output.jpg";
    std::string error;
    Assert::IsTrue(CheckOutput(filename, error));
    Assert::IsFalse(fs::exists(filename));
}

```